

PCIe Gen5 x4/x2/x1 Core 配置差异评审指南

面向 Synopsys PCIe Gen5 Controller/PHY 集成，重点展开 Buffer/Credit/Outstanding 与 BAR/iATU 地址窗口

版本：初版 | 用途：架构配置评审、IP 选型评审、SoC 集成评审、验证计划输入

说明：本文按 PCIe Gen5 通用协议机制和 SoC 集成经验整理。Synopsys IP 不同 release/feature bundle 的具体参数名、可选项和默认值可能不同，最终应以你们拿到的 CoreConsultant/Databook/Integration Guide 为准。文中的量化值用于方案估算和评审，不等同于 IP 约束。

1. 一页结论

- x4/x2/x1 的第一差异是 Max Link Width 和 PHY lane 数量，但真正影响系统性能的是 AXI 带宽、tag/outstanding、buffer/credit、MPS/MRRS 和 NoC/DDR 延迟。
- Gen5 每 lane 有效载荷理论吞吐约 3.94GB/s；x1 约 3.94GB/s，x2 约 7.88GB/s，x4 约 15.75GB/s。实际 TLP 吞吐会受 header、DLLP、ACK/NAK、flow control、read latency 影响。
- Buffer/Credit 不是“越大越好”，关键是要支撑带宽-时延积 BDP。对 MemRd/Completion 吞吐，tag 数和 outstanding read 最敏感；对 MemWr 吞吐，posted data buffer 和 AXI 写入通路最敏感。
- BAR/iATU 不要只按“能访问”来配，要按控制面、数据面、doorbell/mailbox、debug/trace、DMA window 分区规划。x4 带宽更高，建议更早就把地址窗口、权限、QoS 和 SMMU 策略分层。
- MSI/MSI-X 数量与 x4/x2/x1 没有协议绑定，但 x4 通常搭配更多 queue 和 CPU affinity，间接可能需要更多 MSI-X vector 和 GIC LPI。

配置	理论单向有效带宽	推荐 AXI 侧起点	典型风险
Gen5 x1	~3.94GB/s	128b@250-500MHz 通常够	链路够但控制/中断/小包效率低时性能波动明显
Gen5 x2	~7.88GB/s	128b@500MHz 或 256b@250MHz	x2 场景验证容易漏，降宽/恢复 case 要补齐
Gen5 x4	~15.75GB/s	256b@500MHz 级别或更强	AXI/NoC/DDR、tag/outstanding、completion buffer 容易成为瓶颈

2. Buffer、Credit、Outstanding 配置：重点展开

2.1 先区分三个概念

概念	它控制什么	常见瓶颈表现	和 x4/x2/x1 关系
Buffer	本地 Controller 能缓存多少 TLP header/data、completion、replay 内容	吞吐上不去、credit 耗尽、replay 压力大、AXI backpressure 后链路发不动	lane 越多、速率越高，需要更高吸收能力
Credit	PCIe flow control 里对端允许你发多少 PH/PD/NPH/NPD/CplH/CplD	对端不给 credit 时无法继续发某类 TLP	x4 更快消耗 credit，credit 返回延迟不变时更容易打空
Outstanding	本端同时挂起的 read/request/transaction 数量，常和 tag 数绑定	MemRd 吞吐低、读延迟一大就跑不满链路	x4 要跑满一般需要更多 tag 和 outstanding

简单说：写吞吐看 PD buffer 和 AXI 写通路，读吞吐看 Tag/Outstanding 和 Completion buffer。

2.2 量化核心：带宽-时延积 BDP

要跑满链路，系统中必须有足够多的在途数据。粗略估算公式：

$$\text{BDP(Bytes)} = \text{链路有效吞吐(Bytes/s)} \times \text{往返/服务时延(s)}$$

$$\text{需要 outstanding read 数量} \approx \text{BDP} / \text{单个 read completion 返回的数据量}$$

Gen5 不同宽度下，假设端到端 read completion 往返/服务时延为 1us、2us、4us 时，需要的在途数据量如下：

配置	1us BDP	2us BDP	4us BDP
Gen5 x1	~4KB	~8KB	~16KB
Gen5 x2	~8KB	~16KB	~32KB
Gen5 x4	~16KB	~32KB	~64KB

如果 MRRS=512B，且每个 tag 对应一个 512B read request，那么大致需要：

配置	1us 需要 tag	2us 需要 tag	4us 需要 tag
Gen5 x1	8	16	32
Gen5 x2	16	32	64
Gen5 x4	32	64	128

如果 MRRS=1024B，需要 tag 数量约减半：

配置	1us 需要 tag	2us 需要 tag	4us 需要 tag
Gen5 x1	4	8	16
Gen5 x2	8	16	32
Gen5 x4	16	32	64

评审口径：x4 若要靠 MemRd 跑满，tag/outstanding 建议至少按 64 起评估；如果 NoC/DDR/远端响应延迟达到几 us，128/256 才更稳。x1 通常 16-32 个 tag 已经比较够，除非远端响应特别慢。

2.3 Tag、Outstanding、MRRS/MPS 的关系

- Tag 数量限制并发 read request 数量。普通 PCIe 默认 tag 空间较小，开启 Extended Tag 后可支持更多 tag，具体取决于 IP 配置和软件使能。
- MRRS 决定单个 MemRd 最大请求大小。MRRS 越大，同样 BDP 需要的 tag 越少，但 completion 返回可能更大，对 completion buffer 压力更高。
- MPS 决定单个 TLP payload 最大长度。MemWr 吞吐通常更受 MPS 影响；MPS 过小会增加 header 比例和 TLP 数量。
- 对双 SoC 互联，如果是固定拓扑、固定软件栈，建议尽量选择较大的 MPS/MRRS，但必须确认对端、RC/EP、NoC、SMMU 和 DMA 都支持。

目标	偏保守配置	性能配置	注意点
控制面/寄存器访问	MPS 128B, MRRS 128/256B, tag 8-16	不建议过度优化	小包延迟比吞吐重要
DMA 写为主	MPS 256B	MPS 512B 或更大	看 PD buffer、AXI 写带宽、NoC 写吸收能力
DMA 读/远程读为主	MRRS 512B, tag 32-64	MRRS 1024B/2048B, tag 128+	看 completion buffer、read latency、tag 和 AXI 读通路

Gen5 x4 高吞吐	tag 64, outstanding 64	tag 128/256, outstanding 128+	否则 x4 容易跑不满
-------------	------------------------	-------------------------------	-------------

2.4 Credit 分类怎么理解

PCIe flow control 按 TLP 类型和 header/data 资源分 credit，常见为 PH/PD、NPH/NPD、CplH/CplD。

Credit 类型	对应 TLP	主要影响	双 SoC 互联关注点
PH	Posted Header，如 MemWr header	写请求数量	大量 doorbell/小包写会消耗 PH
PD	Posted Data，如 MemWr payload	DMA 写吞吐	x4 MemWr 跑满时 PD 最关键
NPH	Non-Posted Header，如 MemRd header	读请求数量	读 outstanding 越多，NPH 需求越高
NPD	Non-Posted Data，少见	带 data 的 NP 请求	多数内存读场景压力不大
CplH	Completion Header	Completion 数量	小 MRRS/小 completion 会消耗更多 CplH
CplD	Completion Data	读返回数据	远程读/RC 接收 completion 时关键

对于一个方向的数据流，需要看谁发、谁收：

A DMA 写 B 内存：A 消耗 B 给的 PH/PD credit，B 需要 PD 接收/AXI 写吸收。

A DMA 读 B 内存：A 先消耗 B 给的 NPH credit；B 返回 Completion 时消耗 A 给的 CplH/CplD credit，A 需要 completion buffer。

2.5 Buffer 容量的量化估算

Buffer 的真实单位和可配项要看 Synopsys 实现，但评审时可以按“能缓存多少个最大 TLP”估算。

场景	建议估算口径	x1 起点	x2 起点	x4 起点
Posted Data 接收 buffer	至少覆盖若干个 MPS 包 +AXI backpressure 窗口	4-8 个 MPS	8-16 个 MPS	16-32 个 MPS
Completion Data 接收 buffer	覆盖 BDP 和最大 burst completion	8-16KB	16-32KB	32-64KB+
Replay buffer	覆盖发送后未 ACK 的数据窗口	按链路 BDP+DLLP/ACK 延迟	按链路 BDP+余量	x4 需重点看 IP 推荐值
NP request queue	覆盖 outstanding read header	16-32 项	32-64 项	64-128 项
Posted queue	覆盖 doorbell+DMA 写 header	16-32 项	32-64 项	64-128 项

工程判断：如果 x4 配置里 buffer 仍按 x1 级别配置，链路可以训练成 x4，但一旦 AXI 侧有 backpressure 或远端读延迟较大，吞吐会明显掉下来。

2.6 不同业务模式下的配置倾向

业务模式	瓶颈优先级	建议
Mailbox/寄存器访问	延迟、MSI、顺序性	不需要大 buffer；重点是 doorbell、MSI、错误上报和寄存器访问保序
DMA 写为主	PD credit、posted buffer、AXI 写吸收	MPS 可设大；posted data buffer 和 NoC

		写带宽要配足
DMA 读为主	tag/outstanding、MRRS、completion buffer	x4 建议 tag 128 级别评估；MRRS 512B/1024B 起步
双向同时读写	所有 buffer 同时承压	需要验证全双工、completion 和 posted 互相影响、NoC 仲裁 QoS
多队列/多核	queue arbitration、MSI-X、tag 分配	每 queue 的 outstanding 别太小；MSI-X 与 CPU affinity 一起规划

2.7 推荐配置档位

档位	适用	Tag/Outstanding	MPS/MRRS	Buffer 评审目标
轻量 x1	控制面、低速 DMA	16-32	128/256B	几 KB 级 completion, posted 4-8 个 MPS
均衡 x2	中等 DMA、双 SoC 互联基本数据面	32-64	256/512B	16-32KB completion, posted 8-16 个 MPS
性能 x4	高带宽 DMA、远程 memory window	64-128 起, 优先评估 128/256	512B/1024B, 确认双方能力后更大	32-64KB+ completion, posted 16-32 个 MPS
高延迟 x4	远端路径长、NoC/DDR 延迟 us 级	128-256+	1024B/2048B 视能力	BDP 优先, completion 64KB+更稳

3. BAR/iATU 和地址窗口：重点展开

3.1 先分清 Inbound 和 Outbound

方向	含义	典型用途	谁配置
Outbound iATU	本地 AXI 地址 -> PCIe 地址	本地 CPU/DMA 访问对端 BAR 或对端 memory window	本地 RC/EP 的软件或固件
Inbound iATU	PCIe 地址 -> 本地 AXI 地址	对端通过 PCIe 访问本地 SRAM/DDR/register window	被访问端的软件或固件
BAR	对外暴露的 PCIe 地址窗口	让对端枚举/映射后访问本端资源	EP function 配置空间/驱动

在双 SoC 互联里，建议脑子里始终画成两套镜像窗口：

SoC A 访问 SoC B: A 侧 Outbound iATU -> PCIe -> B 侧 BAR/Inbound iATU -> B 侧 AXI 地址
SoC B 访问 SoC A: B 侧 Outbound iATU -> PCIe -> A 侧 BAR/Inbound iATU -> A 侧 AXI 地址

3.2 地址窗口不要只配一个大 BAR

建议至少按功能分区，避免控制面、数据面、doorbell、debug 混在一起。

窗口	建议大小起点	属性建议	用途	原因
Control/Register BAR	64KB-1MB	Non-prefetchable	寄存器、状态、配置	小、强顺序、便于权限控制
Mailbox/Doorbell BAR	4KB-64KB	Non-prefetchable	队列通知、doorbell、mailbox	小写入高频，便于独立 QoS/中断触发
Data Window BAR	16MB-1GB+	Prefetchable, 64-bit	远程内存窗口、DMA buffer	带宽路径，尽量和控制面隔离
MSI/MSI-X Table/PBA	按 vector 数量	通常在 BAR 中映射	MSI-X 表和 pending bit	如果作为 EP 暴露 MSI-X

			array	能力必须规划
Debug/Trace Window	1MB-64MB	按用途	trace buffer、debug CSR	避免污染主数据面

推荐原则：控制面窗口小而明确；数据面窗口大而可扩展；doorbell 单独出来；debug/trace 不要和 data 混用。

3.3 BAR 大小和 iATU region 数量的量化建议

设计目标	BAR/iATU 建议	量化建议
只做 mailbox+少量 DMA	1 个 Control BAR + 1 个 Data BAR	Control 64KB/256KB; Data 16MB-64MB; 每方向 2-4 个 iATU region
双 SoC 远程 memory window	Control + Doorbell + Data	Control 1MB; Doorbell 4KB-64KB; Data 256MB-1GB; 每方向 4-8 个 region
多业务/多安全域	按业务域拆 Data window	每业务域 1 个 region 或 1 组 region; 每方向 8-16 个 region 起评估
虚拟化/SR-IOV 风格	PF/VF 分别规划 BAR	每 VF 独立 BAR/region 会迅速放大资源; 需要先定 VF 数量和每 VF 窗口
外接通用设备	遵循标准枚举资源分配	RC outbound 窗口要能覆盖下游 MMIO aperture, 通常需要 GB 级甚至更大

3.4 64-bit BAR 和 Prefetchable 属性

- Data Window 建议优先用 64-bit BAR，避免 32-bit 地址空间受限，尤其是远程 DDR 窗口、DMA buffer、CXL-like/scale-up 探索场景。
- Control/Register BAR 通常用 Non-prefetchable，防止 CPU/桥片对寄存器访问做不合适的合并、预取或重排。
- Data Window 如果是普通内存语义、允许合并/预取，并且软件映射属性正确，可以考虑 Prefetchable。
- 如果 Data Window 里面混有 doorbell/register，绝对不建议标成 Prefetchable。

3.5 iATU 窗口设计示例

一个适合双 SoC 互联的起步配置可以这样切：

Region	方向	本地 AXI 地址	PCIe 地址/对端 BAR	大小	用途
OB0	A->B	0x8000_0000	B Control BAR	1MB	访问 B 寄存器/mailbox 状态
OB1	A->B	0x8010_0000	B Doorbell BAR	64KB	写 B doorbell
OB2	A->B	0x9000_0000	B Data BAR	256MB	DMA 到 B buffer
IB0	B->A	A Control BAR	A CSR base	1MB	B 访问 A 寄存器
IB1	B->A	A Data BAR	A DDR carveout	256MB	B 访问 A 数据窗口

实际地址要满足 Synopsys iATU region size/alignment 要求。常见约束包括 base/limit 对齐、size 为 2 的幂或最小粒度、region 数量有限、某些 region 类型固定等，必须查你们 release 文档。

3.6 地址窗口和 SMMU/权限/QoS

- 如果 PCIe requester 进入 SoC NoC 后会经过 SMMU/IOMMU，要提前定义 Requester ID/Stream ID 到 SMMU context 的映射。
- Control window 和 Data window 建议分不同权限域：控制面只给安全固件/驱动访问，数据面给 DMA/业务进程访问。
- Doorbell window 建议单独设置 QoS，避免大数据 DMA 把 doorbell 写饿死。

- 远程 memory window 要定义 cacheability、shareability、ordering 和 barrier 语义。PCIe 本质是 I/O 语义，不能默认等同片内 Normal cacheable memory。
- 如果做双向访问，必须确认 A 访问 B 和 B 访问 A 的地址不会形成循环映射或意外反射，避免 P2P 路径死循环和安全漏洞。

3.7 BAR/iATU 常见坑

问题	表现	规避
BAR 太小	驱动映射不够，后期临时扩 BAR 影响软件兼容	初期就分 Control/Data，Data 留扩展
所有功能共用一个 BAR	权限、QoS、debug 困难	Control/Doorbell/Data 拆分
Outbound region 不够	每新增一个远端窗口都要抢 region	按未来业务域预留 region 数量
32-bit BAR 承载大数据窗口	地址空间不够或映射复杂	Data BAR 用 64-bit
Prefetchable 属性乱用	寄存器访问被合并/预取，行为异常	register/doorbell 必须 non-prefetchable
iATU 配置时序错误	枚举前访问失败、link up 后 CPU 读不到	明确枚举前后谁配置哪部分

4. PHY/PIPE、lane 和降宽配置

项目	x1	x2	x4	评审点
Active lane	lane0	lane0-1	lane0-3	固定互联建议 lane 从 0 连续使用
Unused lane	lane1-3	lane2-3	无	未用 lane 不能悬空，要确认 powerdown/tie-off/reset
Lane reversal	通常不用	视走线	常见需求	x4 要验证 lane reversal 和 polarity inversion
Width downgrade	无或 x1	可降 x1	可降 x2/x1	x4 训练成 x1 不等于错误，但要有状态可见性
Equalization	1 条 lane	2 条 lane	4 条 lane	Gen5 x4 最容易某条 lane 拖累整体训练

- x4 配置必须覆盖 x4->x2/x1 降宽训练、Gen5->Gen4/Gen3 降速训练、Recovery 后重新均衡。
- x2 固定互联在板内/芯片间合理，但生态 case 少，验证计划要显式列出 x2 训练和错误恢复。
- 如果使用 x4 PHY 但只使能 x1/x2，要确认未用 lane 的 TX electrical idle、RX termination、PhyStatus/RxValid/RxStatus tie-off。

5. AXI/NoC 接口和吞吐匹配

PCIe 链路带宽只是上限，SoC 侧 AXI/NoC 必须匹配。理论 AXI 带宽公式：

$$\text{AXI 理论带宽(Bytes/s)} = \text{AXI 数据位宽(bit)} / 8 \times \text{AXI 频率(Hz)} \times \text{有效率}$$

AXI 配置	理论带宽	适配建议
128b @250MHz	4GB/s	接近 Gen5 x1 上限，实际效率不足时会瓶颈
128b @500MHz	8GB/s	适合 x2 起步，对 x4 明显不够
256b @250MHz	8GB/s	适合 x2，对 x4 不够
256b @500MHz	16GB/s	接近 Gen5 x4 理论有效带宽
512b @500MHz	32GB/s	x4 有余量，但 NoC/DDR 也要匹配

- 如果 x4 链路实际只跑到 8GB/s 左右，先看 AXI 位宽/频率/效率，再看 tag 和 MRRS。
- NoC 仲裁要考虑读写混合、doorbell 延迟、DMA 大流量下控制面访问是否被饿死。
- DDR 带宽和远端响应延迟直接影响 MemRd 吞吐；远端延迟越大，越需要 tag/outstanding。

6. MSI/MSI-X 与 GIC/ITS 容量

- MSI/MSI-X vector 数量不由 x1/x2/x4 协议决定，而由 function、queue、中断源、CPU affinity 和软件驱动决定。
- 双 SoC 固定互联里，一个 MSI/MSI-X Event 通常映射成一个 GIC LPI；Event 数量和 LPI 数量不是相乘关系。
- 建议轻量方案每对端 function 16-32 个 event；中等多队列方案每对端 function 64 个 LPI 预留；x4 高性能多队列可考虑 128 个 LPI 预留。

场景	建议 vector	GIC/ITS 预留
Control/mailbox	4-8	8-16 LPI
少量 DMA 队列	8-16	16-32 LPI
多队列 DMA	32-64	64-128 LPI
多核 affinity/虚拟化	64-128+	128-256+ LPI

7. Reset/Clock/Power 和低功耗

- 确认 PERST#、Core reset、PHY reset、PIPE reset、app reset 的释放顺序，尤其是双 SoC 互联时谁控制谁。
- 确认 refclk 模式：共参考时钟、SRNS/SRIS、SSC 策略、PLL lock 和 PCLK 来源。
- x1/x2 使用 x4 PHY 时，未用 lane powerdown 不能影响共享 PLL 和 active lane。
- ASPM L0s/L1/L1SS 建议在 bring-up 阶段先关闭，链路稳定后再逐项打开验证。
- 低功耗退出要覆盖 Recovery、重新均衡、MSI/doorbell 路径是否恢复。

8. RC/EP 角色和双 SoC 互联特殊点

- 标准 PCIe 链路两端不能都固定为 RC。两颗相同芯片需要 strap/fuse/boot 配置决定一端 RC、一端 EP，或者支持 dual mode 但上电仲裁必须明确。
- 如果两边都可能作为 RC，要定义枚举责任、PERST# 方向、refclk 提供者、错误恢复策略。
- 如果希望双向访问，EP 侧也需要暴露 BAR/Inbound 窗口，RC 侧也要配置 Outbound 窗口；反方向同理。
- 如果有多个 function，建议 function 职责清晰：PF0 管理/控制，PF1 数据/DMA，PF2 debug/trace 或保留。

9. 验证计划建议

类别	必须覆盖 case
Link 训练	Gen5 x4/x2/x1 正常训练；x4 降 x2/x1；Gen5 失败降 Gen4/Gen3；Recovery 后恢复
Equalization	每 lane preset/coeff、某 lane 质量差、polarity inversion、lane reversal
吞吐	MemWr、MemRd、双向混合；不同 MPS/MRRS；不同 tag/outstanding；AXI backpressure
Flow control	PH/PD/NPH/CplD credit 耗尽、credit 恢复、长延迟 completion

BAR/iATU	枚举前后访问、region 越界、权限错误、32/64-bit BAR、prefetchable 属性
MSI/MSI-X	多个 vector、mask/unmask、doorbell 与 data 并发、GIC LPI 映射
错误	Malformed TLP、UR/CA completion、AER、poisoned TLP、replay、timeout
低功耗	L0s/L1/L1SS 进入退出、PERST#、热复位、link down/up 后重新枚举

10. 最终配置评审 Checklist

- Max Link Width、Max Link Speed 和配置空间 Link Capabilities 是否一致。
- PHY lane 数量、PIPE 位宽、PCLK 架构和 unused lane 处理是否清楚。
- AXI 位宽/频率/CDC 是否能支撑目标 x1/x2/x4 吞吐。
- Tag/outstanding 是否按 BDP 估算过，尤其是 Gen5 x4 读吞吐。
- MPS/MRRS 是否和对端、软件、buffer 能力匹配。
- Posted/Completion/Replay buffer 是否能覆盖 AXI backpressure 和远端响应延迟。
- BAR 是否按 Control/Doorbell/Data/Debug 分区，Data BAR 是否使用 64-bit 并留足空间。
- Outbound/Inbound iATU region 数量是否覆盖未来业务域，region 大小/对齐是否满足 IP 要求。
- SMMU/Requester ID/Stream ID/权限/QoS 是否和地址窗口一起规划。
- MSI/MSI-X vector 和 GIC LPI 是否按 queue/function/event 计算，而不是按 lane 数计算。
- RC/EP 角色、PERST# 方向、refclk 来源和枚举责任是否明确。
- 验证计划是否覆盖降宽、降速、credit 耗尽、AXI backpressure、iATU 越界和 MSI 并发。

附录 A：常用估算公式

Gen5 每 lane 有效吞吐 $\approx 32\text{GT/s} \times 128/130 \div 8 = 3.94\text{GB/s}$

Gen5 xN 有效吞吐 $\approx 3.94\text{GB/s} \times N$

BDP(Bytes) = 吞吐(Bytes/s) $\times$ 往返/服务时延(s)

需要 tag 数 $\approx \text{BDP} / \text{MRRS}$

AXI 理论带宽 = 数据位宽/8 $\times$ 频率 $\times$ 有效率

单颗 SoC 本地 GIC LPI 预留 $\approx$ 对端 Function 数量 $\times$ 每 Function 预留 Event/LPI 数量